

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

PRACTICAL LAB 2

Linux Networking, Scripting, Permissions and Archives

A practical evidence-based laboratory report

Hands-on practice with connectivity tests, listening ports, wget, shell scripts, executable permissions, PATH, package management and tar archives

Student Name	Agbo Daniel Ameh	Registration No.	
Programme/Cohort	OCO Offensive Cyber Operations/Cohort 11	Date	08/08/2026
Instructor	Aminu Idris	Score	/100
Environment	Kali Linux / Ubuntu	Submission	E-Campus

Academic Integrity Declaration

I confirm that the terminal commands, screen shots, outputs, observations and written explanations contained in this laboratory submission are my own practical work done in the prescribed Linux training environment. The evidence is kept from my original practical submission, but is reorganized for clarity and assessment.

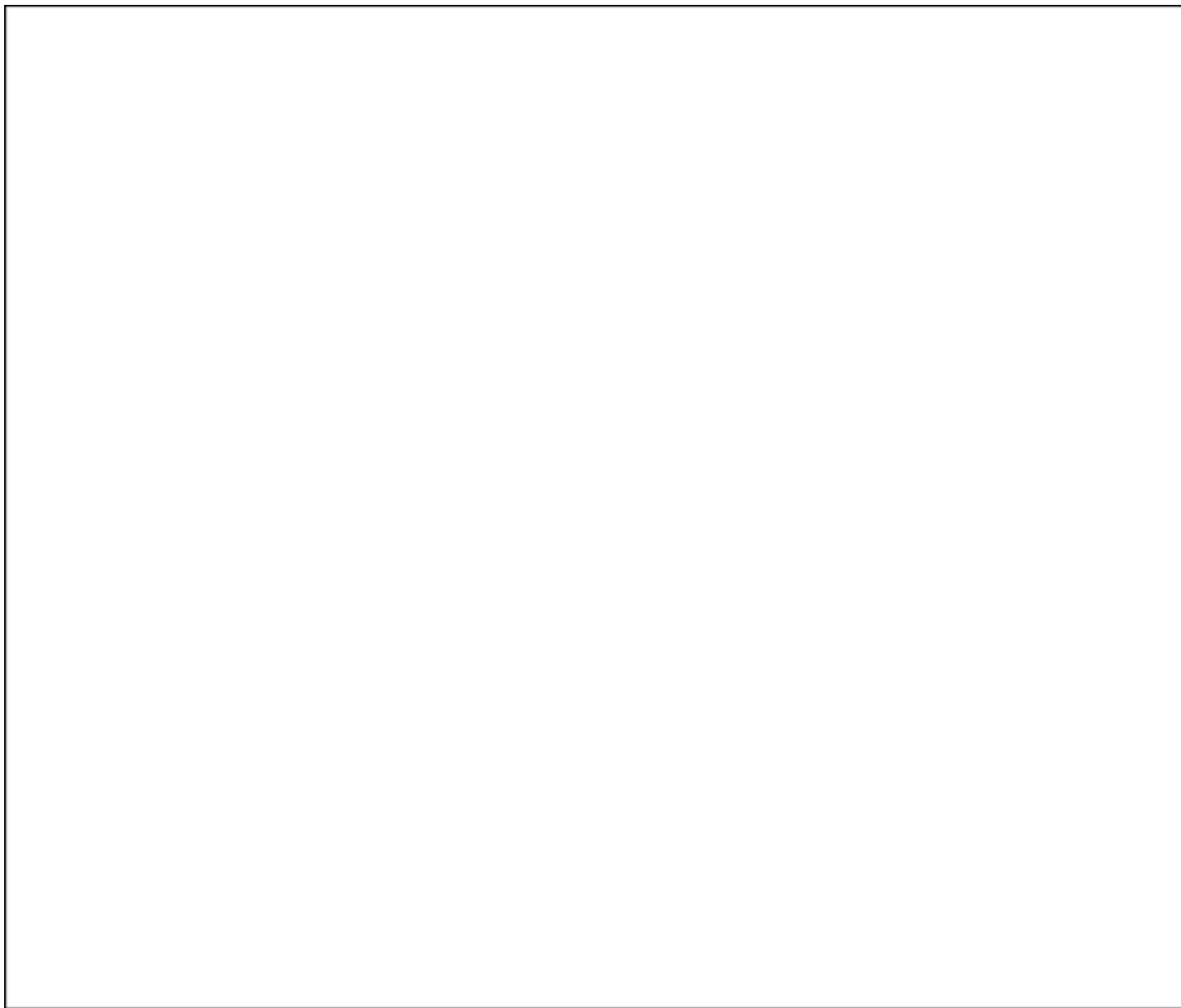
Report contents

1. Task 1 — Test Network Connectivity
2. Task 2 — Inspect Listening TCP Ports
3. Task 3 — Verify a Port on Localhost with nmap
4. Task 4 — Download a File with wget
5. Task 5 — Create and Execute a Shell Script

6. Task 6 — Run Your Script Through PATH
7. Task 7 — Software Update / Installation Practice
8. Task 8 — Create and Extract a tar.gz Archive
9. Submission checklist

Task 1: Test Network Connectivity

The evidence displays the VM network configuration and tests reachability through both a domain name and a public IP address. Both ping commands returned replies in the submitted terminal evidence.



```

(tempadmin@kali)~/icdfa_lab1
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:60:8a:a3 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.220/24 brd 192.168.1.255 scope global dynamic noprefixroute eth0
        valid_lft 1549sec preferred_lft 1549sec
    inet6 2605:59c1:1806:b908:6c23:6d1d:3618:5664/64 scope global temporary dynamic
        valid_lft 216sec preferred_lft 66sec
    inet6 2605:59c1:1806:b908:a00:27ff:fe60:8ae3/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 216sec preferred_lft 66sec
    inet6 fd9f:71da:400c:8:624:7da0:2aa5:1f2c/64 scope global temporary dynamic
        valid_lft 600050sec preferred_lft 81166sec
    inet6 fd9f:71da:400c:8:a00:27ff:fe60:8ae3/64 scope global mngtmpaddr noprefixroute
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe60:8ae3/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(tempadmin@kali)~/icdfa_lab1
$ ping -c 4 example.com
PING example.com (2606:4700:10::6814:179a) 56 data bytes
64 bytes from 2606:4700:10::6814:179a: icmp_seq=1 ttl=53 time=79.9 ms
64 bytes from 2606:4700:10::6814:179a: icmp_seq=2 ttl=53 time=90.6 ms
64 bytes from 2606:4700:10::6814:179a: icmp_seq=3 ttl=53 time=85.8 ms
64 bytes from 2606:4700:10::6814:179a: icmp_seq=4 ttl=53 time=86.6 ms

--- example.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 79.873/85.726/90.623/3.839 ms

(tempadmin@kali)~/icdfa_lab1
$ ping -c 4 example.co
PING example.co (2606:4700:3035::6815:274c) 56 data bytes
64 bytes from 2606:4700:3035::6815:274c: icmp_seq=1 ttl=53 time=103 ms
64 bytes from 2606:4700:3035::6815:274c: icmp_seq=2 ttl=53 time=104 ms
64 bytes from 2606:4700:3035::6815:274c: icmp_seq=3 ttl=53 time=98.8 ms
64 bytes from 2606:4700:3035::6815:274c: icmp_seq=4 ttl=53 time=101 ms

--- example.co ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 98.773/101.765/103.912/1.959 ms

(tempadmin@kali)~/icdfa_lab1
$ ping -c 4 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=58 time=21.3 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=58 time=28.1 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=58 time=19.4 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=58 time=21.6 ms

--- 1.1.1.1 ping statistics ---

```

Figure 1.0 — Network interface/address information and successful domain-name and IP-address connectivity tests.

Written response: A domain-name connection requires name resolution, normally through the Domain Name System (DNS), to translate a hostname such as example.com into one or more IP addresses. A raw IP connection does not require DNS resolution, although the network route and the destination service must still be available.

Task 2: Inspect Listening TCP Ports

ss -lnt lists TCP sockets that are currently listening. The output identifies port 631 bound to 0.0.0.0, meaning the service is listening on the VM's IPv4 interfaces.

```
(tempadmin@kali) - [~/icdfa_lab1]
$ ss -int
State          Recv-Q          Send-Q          Local Address:Port

(tempadmin@kali) - [~/icdfa_lab1]
$ netstat -int
Kernel Interface table
Iface          MTU          RX-OK RX-ERR RX-DRP RX-OVR          TX-OK TX-ERR TX-DRP TX-OVR Flg
eth0            1500          7553    0    15 0          1219    0     8    0 BMRU
lo              65536          21     0     0 0           21     0     0    0 LRU

(tempadmin@kali) - [~/icdfa_lab1]
$
```

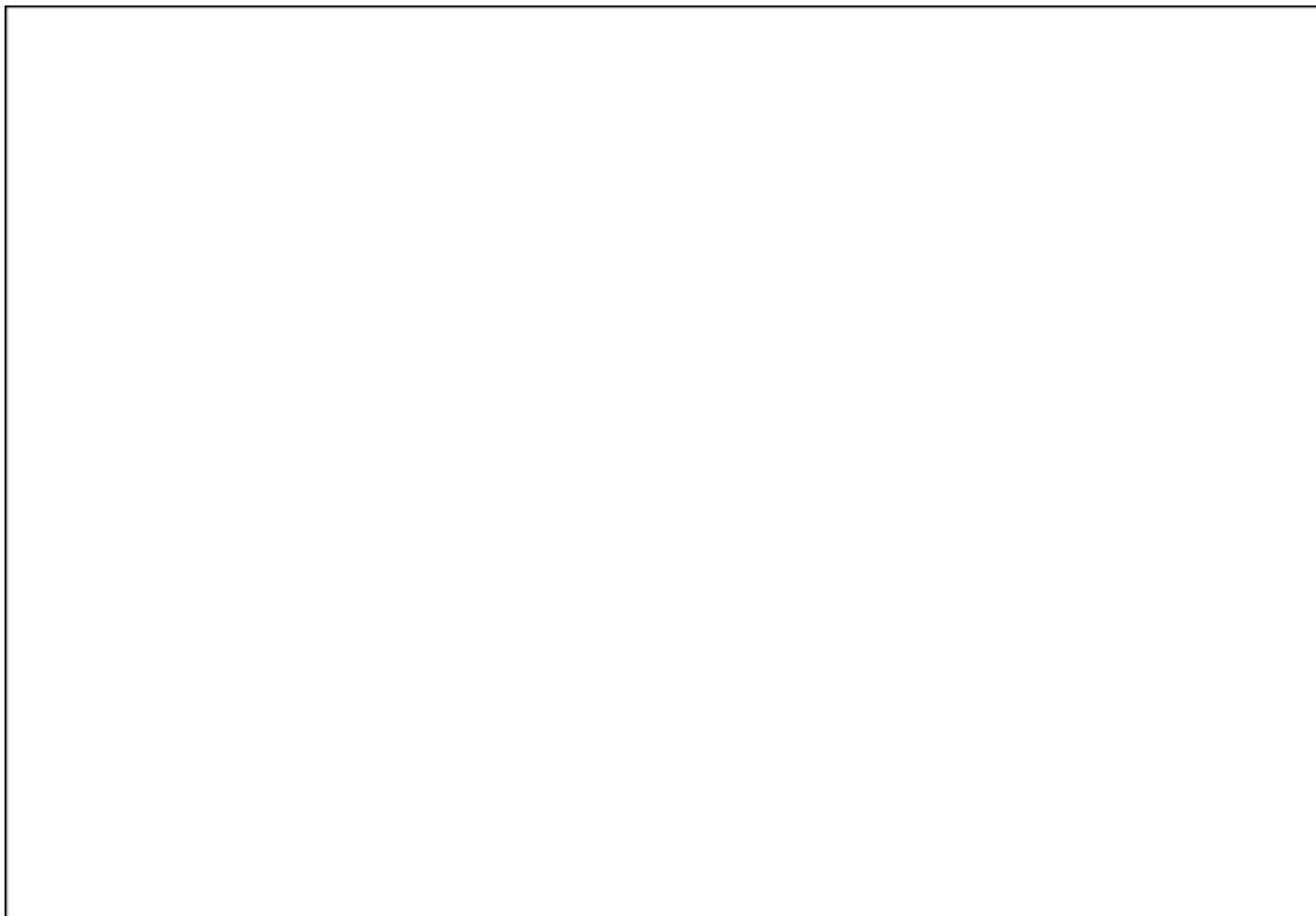
Figure 2.0 — Listening TCP socket output with the identified local address and port.

Identified port: 0.0.0.0:631 (TCP). This is a listening socket shown by the submitted ss output.

Option meanings: -l limits the display to listening sockets; -n keeps addresses and ports numeric instead of resolving names; and -t limits the result to TCP sockets.

Task 3: Verify a Port on Localhost with nmap

The nmap version was checked before two scans were performed against localhost only. This stays within the workbook's authorized-scope requirement.



```

(tempadmin@kali) - [~/icdfa_lab1]
$ nmap --version
Nmap version 7.99 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.8 1 nss-3.6.2 libssh2-1.11.1 libz-1.3.1 libpcap-1.10.6 n
Compiled without:
Available nsock engines: epoll poll select

(tempadmin@kali) - [~/icdfa_lab1]
$ nmap localhost -p 22
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 07:03 -0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000071s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE
22/tcp    closed ssh

Nmap done: 1 IP address (1 host up) scanned in 0.16 seconds

(tempadmin@kali) - [~/icdfa_lab1]
$ nmap localhost -p 21
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 07:04 -0500
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000095s latency).
Other addresses for localhost (not scanned): ::1

PORT      STATE SERVICE
21/tcp    closed ftp

Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds

(tempadmin@kali) - [~/icdfa_lab1]
$

```

Figure 3.0 — nmap version check and localhost scans of TCP ports 22 and 21.

Observed states: Port 22/tcp was reported as closed, and port 21/tcp was reported as closed. No evidence of an open or filtered state is shown in the submitted output.

Written response: A closed port means the host is reachable and nmap received a response, but no application is currently accepting connections on that port. This differs from an open port, where a service is listening, and a filtered port, where a firewall or packet filter prevents nmap from determining the state.

Task 4: Download a File with wget

The permitted example.com page was downloaded into the lab downloads directory, then checked with ls -lh and previewed with head.

```
(tempadmin@kali) - [~/icdfa_lab1]
$ mkdir -p ~/icdfa_lab1/downloads

(tempadmin@kali) - [~/icdfa_lab1]
$ cd ~/icdfa_lab1/downloads/

(tempadmin@kali) - [~/icdfa_lab1/downloads]
$ wget https://example.com/ -O example.html
--2026-09-08 07:14:44-- https://example.com/
Resolving example.com (example.com)... 2606:4700:10::ac42:93f3, 2606:4700:10::6814:179a, 104.20.23.154, ...
Connecting to example.com (example.com)|2606:4700:10::ac42:93f3|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/html]
Saving to: 'example.html'

example.html                               [ <=> ]           559  --.-KB/s  in
2026-09-08 07:14:45 (9.16 MB/s) - 'example.html' saved [559]

(tempadmin@kali) - [~/icdfa_lab1/downloads]
$ ls -lh example.html
-rw-rw-r-- 1 tempadmin tempadmin 559 Aug 29 23:11 example.html

(tempadmin@kali) - [~/icdfa_lab1/downloads]
$ head example.html
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width, height=device-height, initial-scale=1"><meta charset="utf-8"><style>body{background-color:#eee;width:60vw;margin:15vh auto;font-family:sans-serif}</style><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><a href="http://example.com">http://example.com</a></div></body></html>

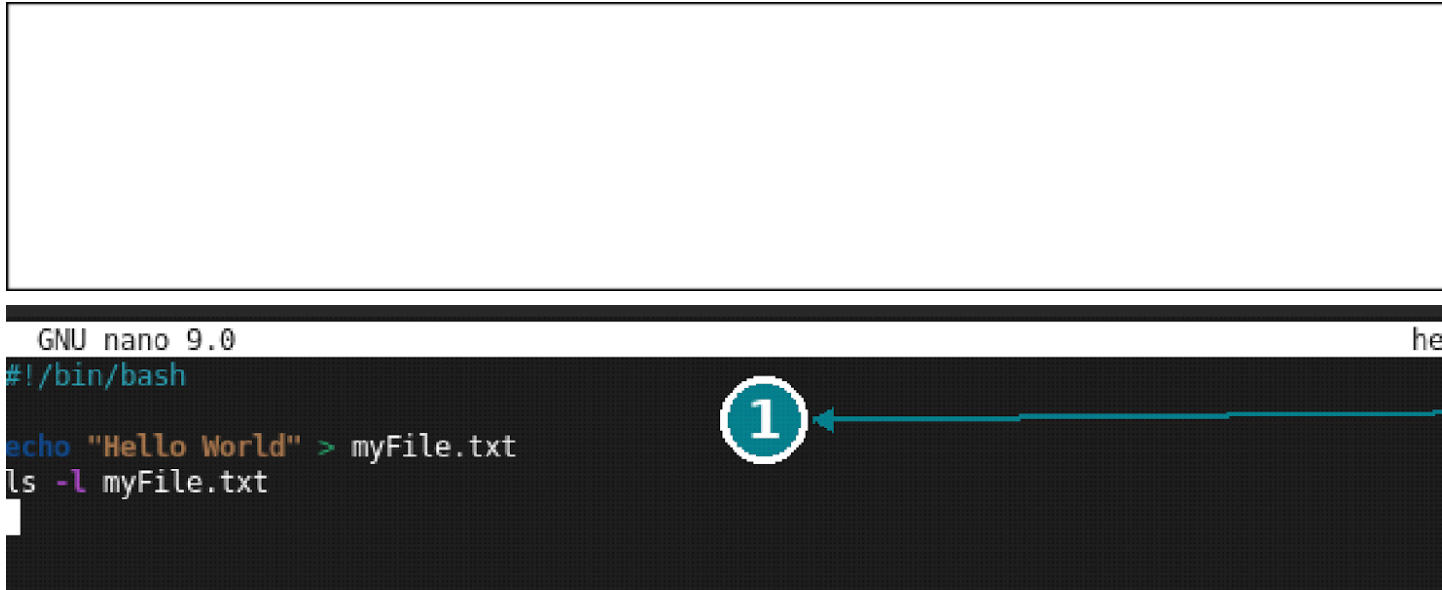
(tempadmin@kali) - [~/icdfa_lab1/downloads]
$
```

Figure 4.0 — wget download, file-size verification, and the beginning of the downloaded HTML file.

Verification: The HTTP request completed successfully, example.html was created, and head displayed the beginning of the Example Domain HTML response.

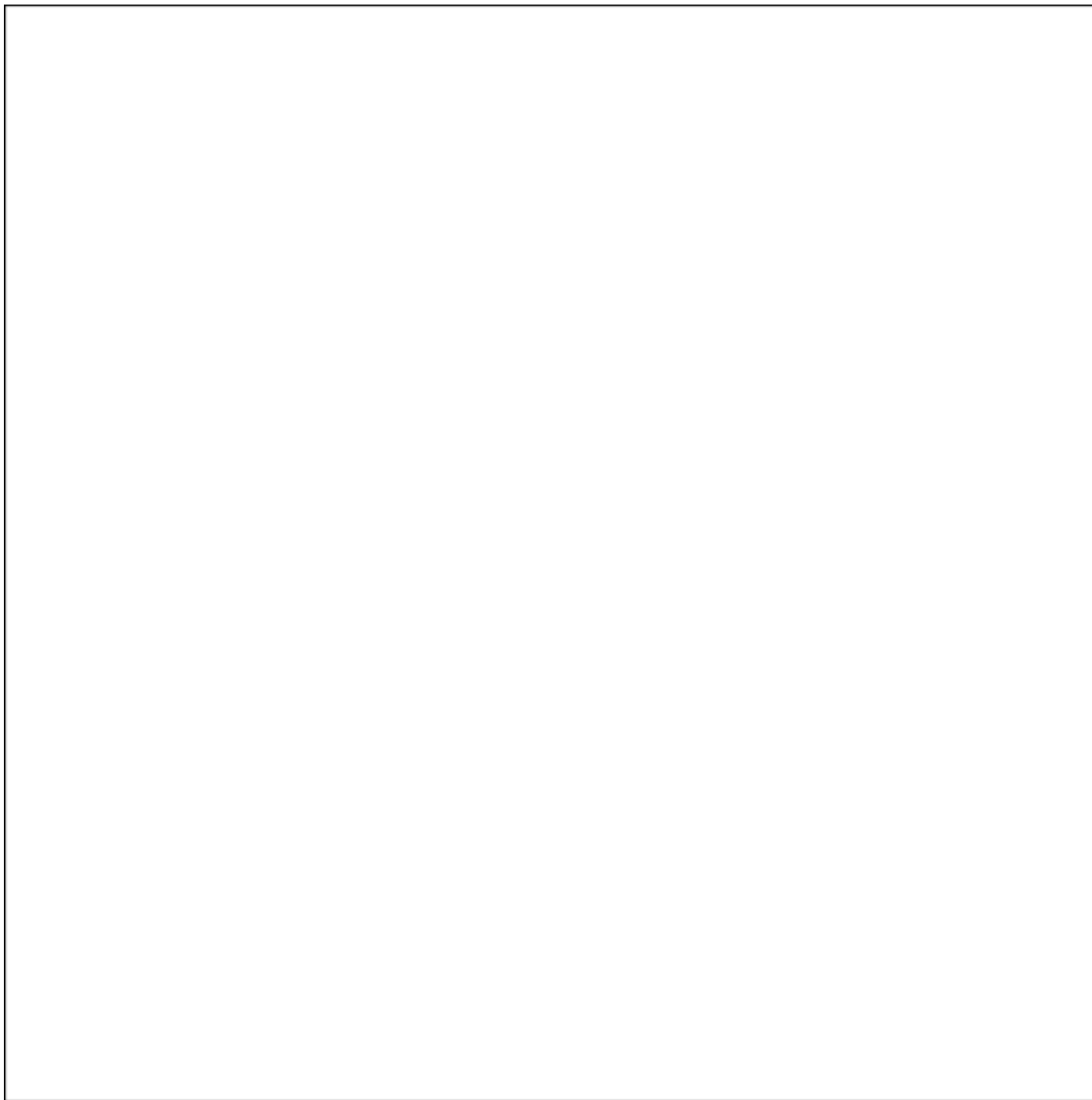
Task 5: Create and Execute a Shell Script

The script was created with a shebang, inspected, made executable, run, and verified through the generated myFile.txt output. The evidence intentionally shows permissions before and after chmod.



```
GNU nano 9.0
#!/bin/bash
echo "Hello World" > myFile.txt
ls -l myFile.txt
```

Figure 5.0 — Script source entered in nano.



```
(tempadmin@kali)-[~/icdfa_lab1]
$ mkdir -p ~/icdfa_lab2/scripts

(tempadmin@kali)-[~/icdfa_lab1]
$ cd ~/icdfa_lab2/scripts/

(tempadmin@kali)-[~/icdfa_lab2/scripts]
$ nano hello_icdfa.sh

(tempadmin@kali)-[~/icdfa_lab2/scripts]
$ cat hello_icdfa.sh
#!/bin/bash

echo "Hello World" > myFile.txt
ls -l myFile.txt

(tempadmin@kali)-[~/icdfa_lab2/scripts]
$ ls -l hello_icdfa.sh
-rw-rw-r-- 1 tempadmin tempadmin 63 Sep  8 07:29 hello_icdfa.sh

(tempadmin@kali)-[~/icdfa_lab2/scripts]
$ chmod +x hello_icdfa.sh

(tempadmin@kali)-[~/icdfa_lab2/scripts]
$ ls -l hello_icdfa.sh
-rwxrwxr-x 1 tempadmin tempadmin 63 Sep  8 07:29 hello_icdfa.sh

(tempadmin@kali)-[~/icdfa_lab2/scripts]
$ ./hello_icdfa.sh
-rw-rw-r-- 1 tempadmin tempadmin 12 Sep  8 07:31 myFile.txt

(tempadmin@kali)-[~/icdfa_lab2/scripts]
$ cat myFile.txt
Hello World

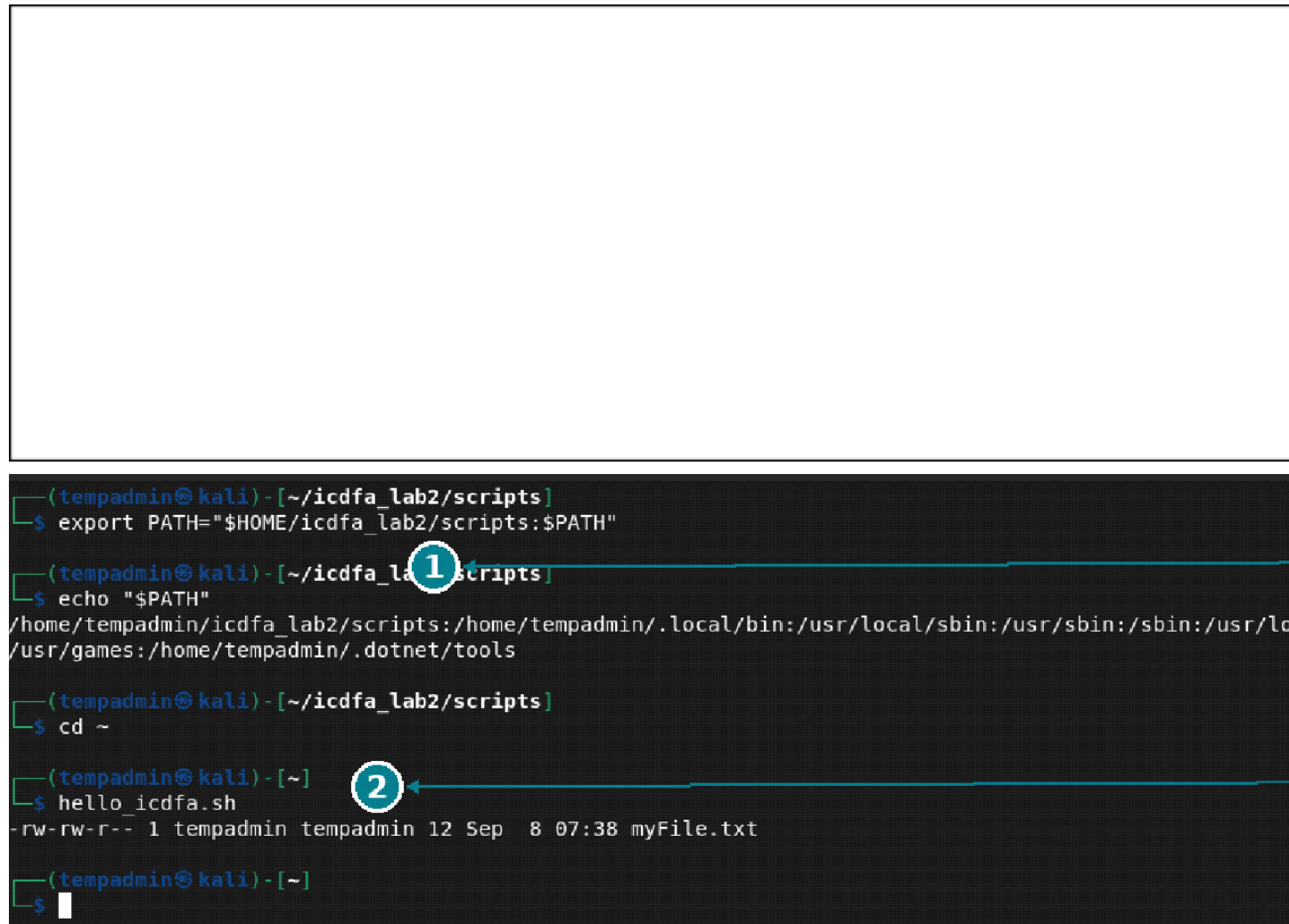
(tempadmin@kali)-[~/icdfa_lab2/scripts]
$
```

Figure 6.0 — Script contents, permissions before and after chmod, execution, and generated myFile.txt.

Written response: The line `#!/bin/sh` is the shebang. It tells the operating system to run the file using the `/bin/sh` shell interpreter when the script is executed as a program. This makes the intended interpreter explicit instead of relying on the user's interactive shell.

Task 6: Run Your Script Through PATH

The script directory was added to PATH for the current shell session. The script was then executed from the home directory without `./` or its full path, demonstrating command lookup through PATH.



```
(tempadmin@kali) - [~/icdfa_lab2/scripts]
$ export PATH="$HOME/icdfa_lab2/scripts:$PATH"

(tempadmin@kali) - [~/icdfa_lab2/scripts]
$ echo "$PATH"
/home/tempadmin/icdfa_lab2/scripts:/home/tempadmin/.local/bin:/usr/local/sbin:/usr/sbin:/sbin:/usr/local/bin:/usr/games:/home/tempadmin/.dotnet/tools

(tempadmin@kali) - [~/icdfa_lab2/scripts]
$ cd ~

(tempadmin@kali) - [~]
$ hello_icdfa.sh
-rw-rw-r-- 1 tempadmin tempadmin 12 Sep  8 07:38 myFile.txt

(tempadmin@kali) - [~]
$
```

The terminal screenshot shows a Kali Linux shell session. The user is in the directory `~/icdfa_lab2/scripts`. They run `export PATH="$HOME/icdfa_lab2/scripts:$PATH"` to add the current directory to the PATH. Then, they run `echo "$PATH"` to verify the PATH, which shows the current directory at the beginning of the list. Next, they run `cd ~` to move to the home directory. Finally, they run `hello_icdfa.sh`, which executes successfully. The output of the script is a file listing: `-rw-rw-r-- 1 tempadmin tempadmin 12 Sep 8 07:38 myFile.txt`. The prompt returns to `(tempadmin@kali) - [~]` with a cursor on the next line.

Figure 7.0 — Temporary PATH extension and successful script execution from a different working directory.

Observation: After the export command, the shell located `hello_icdfa.sh` in `~/icdfa_lab2/scripts` and executed it from the home directory. The resulting `myFile.txt` listing confirms that the script ran successfully.

Task 7: Software Update / Installation Practice

The package-management evidence shows an authorized apt update attempt and a check for terminator. The report records the actual outcome rather than implying that installation succeeded.

```
(tempadmin@kali) - [~]
$ sudo apt update
[sudo] password for tempadmin:
Notice: It seems that you don't have any APT data sources configured.
Notice: You won't be able to 1 update your system or install new packages.
Notice: For more information, please refer to the online documentation at:
Notice: https://www.kali.org/docs/general-use/kali-apt-sources/
All packages are up to date.

(tempadmin@kali) - [~]
$ dpkg -s terminator 2>/dev/null | head

(tempadmin@kali) - [~]
$ sudo apt install terminator
Error: Unable to locate package 2 terminator

(tempadmin@kali) - [~]
$
```

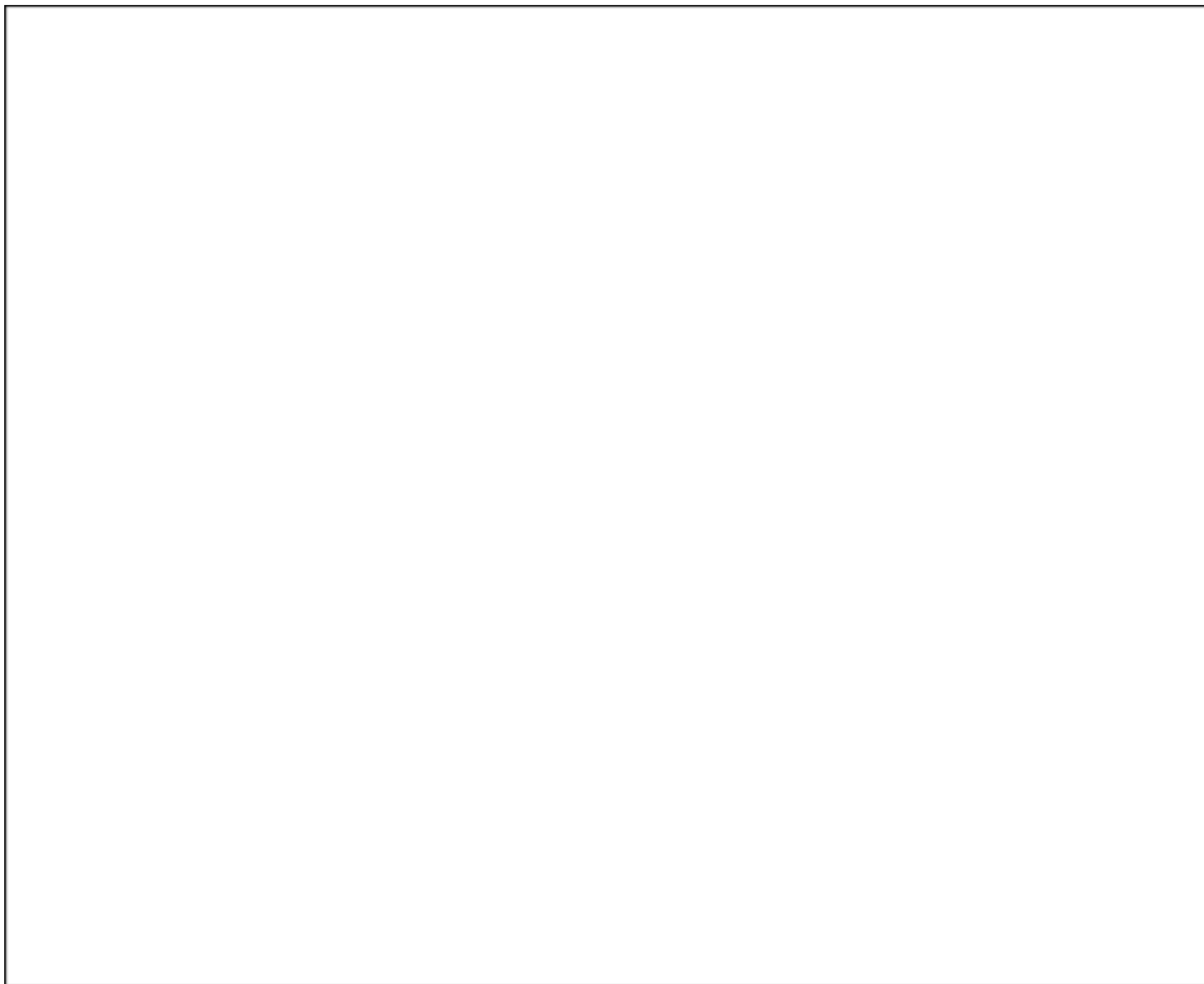
Figure 8.0 — apt update output, package-status check, and the attempted terminator installation result.

Outcome: The package lists were refreshed successfully, but the package-status check indicated that terminator was not installed. The subsequent installation command did not complete because the package could not be located in the configured repositories. No unrelated packages or repository settings were changed.

Command meanings: `sudo apt update` refreshes package metadata; `dpkg -s terminator` checks the local package database; and `sudo apt install terminator` requests installation from the configured repositories.

Task 8: Create and Extract a tar.gz Archive

The scripts directory was compressed into `scripts_backup.tar.gz`, the archive was checked, and its contents were extracted into `extracted`. The final find output verifies the extracted script and generated text file.



```
(tempadmin@kali) - [~]
$ cd ~/icdfa_lab2

(tempadmin@kali) - [~/icdfa_lab2]
$ tar czvf scripts_backup.tar.gz scripts
scripts/
scripts/hello_icdfa.sh
scripts/myFile.txt
1

(tempadmin@kali) - [~/icdfa_lab2]
$ ls -lh scripts_backup.tar.gz
-rw-rw-r-- 1 tempadmin tempadmin 246 Sep  8 07:53 scripts_backup.tar.gz

(tempadmin@kali) - [~/icdfa_lab2]
$ mkdir -p extracted

(tempadmin@kali) - [~/icdfa_lab2]
$ tar -xzvf scripts_backup.tar.gz -C extracted
scripts/
scripts/hello_icdfa.sh
scripts/myFile.txt
2

(tempadmin@kali) - [~/icdfa_lab2]
$ find extracted -maxdepth 3 -type f -print
extracted/scripts/hello_icdfa.sh
extracted/scripts/myFile.txt

(tempadmin@kali) - [~/icdfa_lab2]
$ 3
```

Figure 9.0 — tar.gz creation, archive verification, extraction, and final file listing.

Option meanings: In tar -czvf, -c creates an archive, -z applies gzip compression, -v prints processed files, and -f supplies the archive filename. In tar -xzvf, -x extracts, -z handles gzip compression, -v prints extracted files, and -f identifies the archive.